PYTHON FUNDAMENTALS

200 Practice Questions

A structured roadmap from basic syntax to files, modules, and small integrated projects.

How to use this workbook

• Work through the phases in order, and try each problem before looking up a solution.

• Write complete programs, test normal and edge cases, and explain why your code works.

• Treat the questions as practice prompts—not a checklist to rush through. Revisit problems you cannot solve independently.

• This roadmap focuses on Python fundamentals and the standard library. Larger third-party tools such as NumPy, Pandas, SciPy, and Django can be studied separately.

12 phases • 200 questions • beginner to core Python proficiency

Phase 1 — Input, Output, Variables, and Arithmetic

1. Write a program that asks the user for their name and age, then prints a friendly message containing both values.

2. Ask the user to enter two numbers, then display their sum, difference, product, quotient, and remainder where applicable.

3. Write a program that converts a temperature from Celsius to Fahrenheit using the standard conversion formula.

4. Ask the user for the length and width of a rectangle, then calculate and display its area and perimeter.

5. Write a program that calculates the area and circumference of a circle from a radius supplied by the user.

6. Ask the user for a number of seconds and convert it into hours, minutes, and remaining seconds.

7. Write a program that swaps the values of two variables, first using a temporary variable and then without one.

8. Ask the user for a bill amount and a percentage, then calculate the tip and the final amount to pay.

9. Write a program that converts a total number of days into weeks and remaining days.

10. Ask the user for a three-digit integer and calculate the sum of its digits.

Phase 2 — Conditions and Boolean Logic

11. Ask the user for an integer and determine whether it is positive, negative, or zero.

12. Write a program that checks whether an integer is even or odd.

13. Ask the user for two numbers and display the larger one, or explain that they are equal.

14. Write a program that finds the largest of three numbers without using the built-in max() function.

15. Ask the user for a year and determine whether it is a leap year.

16. Write a program that checks whether a given character is a vowel or a consonant.

17. Ask the user for a student's marks and print the grade according to a clearly defined grading scale.

18. Write a program that checks whether a number is divisible by both 5 and 11.

19. Ask the user for three side lengths and determine whether they can form a valid triangle.

20. If the three side lengths form a valid triangle, classify it as equilateral, isosceles, or scalene.

21. Write a program that calculates an electricity bill using different rates for different consumption ranges.

22. Ask the user for a username and password and display whether the supplied credentials match predefined values.

23. Write a program that checks whether a given character is an uppercase letter, lowercase letter, digit, or special character.

24. Ask the user for a number and determine whether it lies within a specified inclusive range.

25. Write a simple calculator that asks for two numbers and an operator, then performs the requested operation while handling division by zero.

Phase 3 — Loops, Number Logic, and Series

26. Use a loop to print every integer from 1 through 100.

27. Use a loop to print all integers from 100 down to 1.

28. Ask the user for an integer n and calculate the sum of all integers from 1 through n.

29. Ask the user for an integer n and calculate n factorial using a loop.

30. Print the multiplication table for a number entered by the user.

31. Display all even numbers between two limits entered by the user.

32. Display all odd numbers between two limits entered by the user.

33. Count how many digits are present in a positive integer without converting it to a string.

34. Reverse the digits of an integer using arithmetic and a loop.

35. Determine whether an integer is a palindrome without converting it to a string.

36. Calculate the sum of the digits of an integer using a loop.

37. Calculate the product of the digits of an integer using a loop.

38. Determine whether a number is prime by checking its possible divisors.

39. Print every prime number within a range supplied by the user.

40. Find the greatest common divisor (GCD) of two positive integers using a loop.

41. Find the least common multiple (LCM) of two positive integers using their GCD or an iterative method.

42. Generate the first n terms of the Fibonacci sequence.

43. Ask the user for a number and determine whether it belongs to the Fibonacci sequence.

44. Check whether a number is an Armstrong number by raising each digit to the number of digits and adding the results.

45. Print all Armstrong numbers within a user-specified range.

46. Check whether a number is a perfect number by adding its proper divisors.

47. Print all perfect numbers within a user-specified range.

48. Generate the series 1, 1/2, 1/3, ... up to n terms and calculate its sum.

49. Calculate the sum of the series 1 + 2 + 3 + ... + n using a loop, then compare it with the formula result.

50. Keep asking the user for numbers until they enter a sentinel value, then display the count, total, and average of the entered numbers.

Phase 4 — Nested Loops and Patterns

51. Use nested loops to print a square of asterisks with a side length entered by the user.

52. Print a right-angled triangle of asterisks whose height is specified by the user.

53. Print an inverted right-angled triangle of asterisks.

54. Print a centered pyramid of asterisks for a height entered by the user.

55. Print an inverted centered pyramid of asterisks.

56. Print a hollow square pattern using asterisks, leaving the inside empty.

57. Print a number triangle in which each row contains the numbers from 1 up to that row's number.

58. Print a triangle in which each row repeats the row number across the row.

59. Print a Floyd's triangle containing consecutive integers starting from 1.

60. Print a diamond pattern of asterisks using nested loops.

61. Print a multiplication table grid from 1 × 1 through 10 × 10.

62. Print a checkerboard pattern using two alternating characters.

Phase 5 — Lists and List Operations

63. Create a list of numbers entered by the user, then display the list, its length, and its sum.

64. Find the largest and smallest values in a list without using max() or min().

65. Calculate the average of the values in a list without using statistics.mean().

66. Count how many even and odd numbers appear in a list.

67. Create a new list containing only the positive numbers from an existing list.

68. Remove duplicate values from a list while preserving the order of their first appearance.

69. Reverse a list without using the built-in reverse() method or slicing.

70. Check whether a specified value appears in a list and report its index if it does.

71. Count how many times a particular value occurs in a list.

72. Find the second-largest distinct value in a list and handle cases where it does not exist.

73. Merge two lists and produce a sorted result without modifying either original list.

74. Separate a list of numbers into two lists: one for even numbers and one for odd numbers.

75. Rotate a list to the right by k positions, handling values of k larger than the list length.

76. Move every zero in a list to the end while preserving the order of the non-zero elements.

77. Find the common elements between two lists without including duplicates in the result.

78. Find the elements that appear in one list but not in another.

79. Split a list into chunks of a specified maximum size.

80. Flatten a one-level nested list into a single list.

81. Find all pairs of numbers in a list whose sum equals a target value.

82. Copy a list and demonstrate the difference between assigning another reference and creating an independent shallow copy.

Phase 6 — Strings and Text Processing

83. Ask the user for a sentence and count the number of characters, excluding or including spaces as separate results.

84. Count the number of words in a sentence, handling repeated spaces between words.

85. Reverse a string without using slicing.

86. Determine whether a string is a palindrome after ignoring case and non-alphanumeric characters.

87. Count the vowels and consonants in a string.

88. Count the uppercase letters, lowercase letters, digits, and whitespace characters in a string.

89. Build a frequency table showing how many times each character appears in a string.

90. Find the first non-repeating character in a string, or report that every character repeats.

91. Remove duplicate characters from a string while preserving their original order.

92. Replace every occurrence of a specified substring with another substring.

93. Check whether two strings are anagrams after ignoring spaces and letter case.

94. Find the longest word in a sentence and handle ties consistently.

95. Capitalize the first letter of every word without relying entirely on title().

96. Convert a sentence into a list of words and then join the words with a chosen separator.

97. Count how many times a word appears in a sentence without counting it as part of a longer word.

98. Check whether a string contains only digits without using isdigit().

99. Validate a simple password using rules for minimum length and required character types.

100. Extract all integers from a string containing both text and numbers.

101. Compress a string by replacing consecutive repeated characters with each character followed by its count.

102. Format a person's name and a decimal amount neatly using f-strings and specified decimal precision.

Phase 7 — Tuples, Sets, and Dictionaries

103. Store a person's name, age, and city in a tuple, then unpack the values into separate variables.

104. Write a function that returns both the minimum and maximum values from a collection as a tuple.

105. Given two sets, display their union, intersection, difference, and symmetric difference.

106. Remove duplicates from a list by using a set, then explain why the original order may not be preserved.

107. Check whether one set is a subset of another and whether two sets are disjoint.

108. Create a dictionary that maps student names to marks and print each student's name with their marks.

109. Add a new key-value pair to a dictionary, update an existing value, and safely handle a missing key.

110. Count the frequency of each word in a sentence using a dictionary.

111. Build a phone book that allows the user to add, search for, update, and delete contacts.

112. Merge two dictionaries and define what should happen when both contain the same key.

113. Find the key associated with the largest value in a dictionary.

114. Sort a dictionary's items by key and then, in a separate result, by value.

115. Group a list of words by their first letter using a dictionary.

116. Create a dictionary from two lists, one containing keys and the other containing values, while handling unequal lengths.

117. Use a dictionary to count the frequency of each character in a string.

118. Represent a shopping cart as a dictionary of item names and quantities, then calculate the total quantity of items.

119. Use a set to find students who enrolled in both of two courses and students enrolled in only one course.

120. Create a nested dictionary for students and their subject marks, then calculate each student's average.

Phase 8 — Functions, Recursion, and Scope

121. Write a function that accepts two numbers and returns their sum, difference, product, and quotient.

122. Write a function that returns the factorial of a non-negative integer and validates its input.

123. Write a function that checks whether a number is prime and returns a Boolean result.

124. Write a function that accepts a list of numbers and returns its average, handling an empty list safely.

125. Write a function that converts a temperature between Celsius and Fahrenheit based on a parameter.

126. Create a function with a default argument and demonstrate calling it both with and without that argument.

127. Write a function that accepts any number of positional arguments and returns their sum.

128. Write a function that accepts keyword arguments and prints each key-value pair neatly.

129. Write a function that returns multiple values and unpack those values when calling it.

130. Write a recursive function that calculates the factorial of a number.

131. Write a recursive function that returns the nth Fibonacci number.

132. Write a recursive function that calculates the sum of the digits of a positive integer.

133. Write a recursive function that reverses a string.

134. Implement a recursive binary search on a sorted list and return the target index or -1.

135. Demonstrate the difference between a local variable and a global variable in a small program.

136. Write a function that accepts another function as an argument and applies it to a list of values.

137. Create a lambda function and use it as the key for sorting a list of tuples.

138. Write a function that uses a docstring, type hints, and clear parameter names, then call it with valid and invalid inputs.

Phase 9 — Core Python Features and Idioms

139. Use a list comprehension to create the squares of all even numbers in a given range.

140. Use a dictionary comprehension to map each number in a range to its square.

141. Use a set comprehension to collect the distinct vowels found in a sentence.

142. Use enumerate() to print each item in a list alongside its index, starting the index at 1.

143. Use zip() to combine two lists of equal length into pairs and handle lists of unequal length.

144. Use sorted() with a custom key to sort a list of words by length and then alphabetically.

145. Use any() and all() to test whether a list satisfies two different conditions.

146. Use map() and filter() to transform and select values from a list, then solve the same task with a comprehension.

147. Write a generator function that yields the first n square numbers one at a time.

148. Create an iterator or generator that produces even numbers within a specified range.

149. Use unpacking to assign the first item, last item, and remaining middle items of a list to separate variables.

150. Write a function that accepts a variable number of positional and keyword arguments using *args and **kwargs.

151. Use try/except/else/finally to handle a conversion that may fail, and explain when each block executes.

152. Raise a ValueError when a function receives an invalid input and catch it at the point where the program can respond appropriately.

153. Write a program that uses assertions to check important assumptions about a calculation.

154. Demonstrate how shallow copying differs from deep copying when a list contains nested lists.

155. Create a Python module with a reusable function and protect its example-running code with if __name__ == '__main__'.

Phase 10 — Files, Exceptions, JSON, and Regular Expressions

156. Create a text file from Python, write several lines to it, close it safely, and then read the contents back.

157. Ask the user for a filename and append new text to the file without deleting its existing contents.

158. Read a text file line by line and count its total lines, words, and characters.

159. Copy the contents of one text file into another while handling the possibility that the source file does not exist.

160. Search a text file for a given word and report the line numbers on which it appears.

161. Replace a specified word throughout a text file and save the result to a new file.

162. Create a directory if it does not already exist, then create a file inside it using pathlib.

163. Rename a file and delete a file after checking that the intended path exists.

164. Handle FileNotFoundError and PermissionError when opening a file, and display a helpful message to the user.

165. Store a dictionary in a JSON file and load it back into Python while handling malformed JSON.

166. Create a small contact book that loads contacts from JSON, allows updates, and saves the updated data.

167. Read a CSV file using Python's csv module and print its rows in a readable format.

168. Read a CSV file containing numeric values and calculate a total and average for a chosen column.

169. Use a regular expression to find every email-like address in a block of text.

170. Use a regular expression to extract all integer and decimal numbers from a string.

171. Validate a date string in a specified format using a regular expression, while recognizing that pattern matching alone does not prove a date is real.

172. Write a program that asks for a filename, catches likely file errors, and logs a useful error message without crashing unexpectedly.

Phase 11 — Object-Oriented Programming

173. Create a Student class with attributes for a name and marks, plus a method that displays the student's details.

174. Create a Rectangle class whose methods calculate its area and perimeter.

175. Create a BankAccount class with deposit and withdrawal methods that prevent invalid transactions.

176. Write a constructor that assigns default values when optional arguments are not supplied.

177. Use instance attributes and class attributes in a class, then demonstrate how they differ.

178. Add __str__() to a class so that its objects display a readable description when printed.

179. Create a class with a method that updates an object's attributes while validating the new values.

180. Use inheritance to create a SavingsAccount class derived from a BankAccount class.

181. Override a parent-class method in a child class and demonstrate which implementation is called.

182. Use super() in a child class to reuse initialization or behaviour from its parent class.

183. Create an abstract base class for shapes and implement area calculations in two concrete subclasses.

184. Demonstrate polymorphism by calling the same method on objects of different classes.

185. Use encapsulation conventions and properties to control access to an object's attribute.

186. Implement __len__() or __eq__() for a custom class and demonstrate the behaviour.

187. Represent a library book and its availability with a class, including methods to borrow and return the book.

188. Create a simple inventory system using Product objects and methods to add stock, sell stock, and report remaining stock.

189. Use composition by creating a Car object that contains an Engine object.

190. Build a small class-based expense tracker that stores transactions and calculates the total expenses.

Phase 12 — Modules, Packages, and Integration Projects

191. Import the math module and write a program that uses several of its functions to solve a practical calculation.

192. Use random to create a number-guessing game that gives feedback and counts the number of attempts.

193. Use datetime to calculate the number of days between two dates entered by the user.

194. Use time.perf_counter() to measure how long a chosen operation takes, and explain why timing can vary.

195. Use os or pathlib to list files in a directory and filter the results by file extension.

196. Use shutil to copy a file to a backup location and handle errors if the source or destination is invalid.

197. Use statistics to calculate the mean and median of a dataset, and explain what happens when the dataset is empty.

198. Use sys.argv to accept a value from the command line and display a helpful usage message when the argument is missing.

199. Split reusable functions into a separate Python module and import them into a main program.

200. Build a command-line expense tracker that reads transactions from CSV, calculates summary statistics, and saves a JSON report.